

AnyDoc: A Hybrid Retrieval-Augmented Generation System with Query Rewriting and Cross-Encoder Reranking

Independent project · Source: github.com/Tarunvarma07/AnyDoc_chat

Abstract

We present AnyDoc, a document question-answering system that combines dense and sparse retrieval, cross-encoder reranking, and LLM-based query rewriting in a single pipeline. Unlike single-retriever RAG implementations, AnyDoc fuses ChromaDB dense vector search with a BM25 sparse index via Reciprocal Rank Fusion (RRF), then re-scores the fused candidate set with a cross-encoder before generation. A lightweight query rewriting stage expands the user's question into a keyword-enriched search query prior to retrieval, independent of the original question used for answer generation. The ingestion path is hardened against Server-Side Request Forgery (SSRF) via explicit private-IP and cloud-metadata blocking, redirect re-validation, and file magic-byte verification. We evaluate the retrieval stack on a 105-document adversarial benchmark containing eight exact-match facts buried among aggressive semantic distractors, and report hit-rate results for dense-only versus hybrid retrieval. We discuss where each component is expected to matter most at scale and outline the system's current limitations.

1. Introduction

Retrieval-Augmented Generation (RAG) systems ground large language model outputs in retrieved documents, reducing hallucination and enabling up-to-date, source-attributable answers (Lewis et al., 2020). A common failure mode of RAG implementations that rely solely on dense vector similarity is poor recall on queries that hinge on exact tokens - identifiers, error codes, version strings - which dense embeddings can under-weight relative to broader semantic similarity. AnyDoc addresses this by combining dense retrieval with a classical sparse retriever (BM25) and fusing the two result sets with Reciprocal Rank Fusion (Cormack et al., 2009), then applying a cross-encoder reranker (Nogueira and Cho, 2019) to the fused candidate pool before generation.

This report documents the system's architecture, evaluates the retrieval stack against an adversarial benchmark corpus, and situates the design against prior work. The contributions are:

- A hybrid dense + sparse retriever with RRF fusion and an event-driven index that stays consistent as documents are added.
- A cross-encoder reranking stage that widens the pre-fusion candidate pool and re-scores it jointly against the query.
- An LLM query-rewriting stage that decouples the retrieval query from the generation query.
- SSRF- and MIME-hardened ingestion for URL and file sources.
- A reproducible, adversarial evaluation harness comparing dense-only and hybrid retrieval hit rates.

2. System Architecture

System Component Diagram

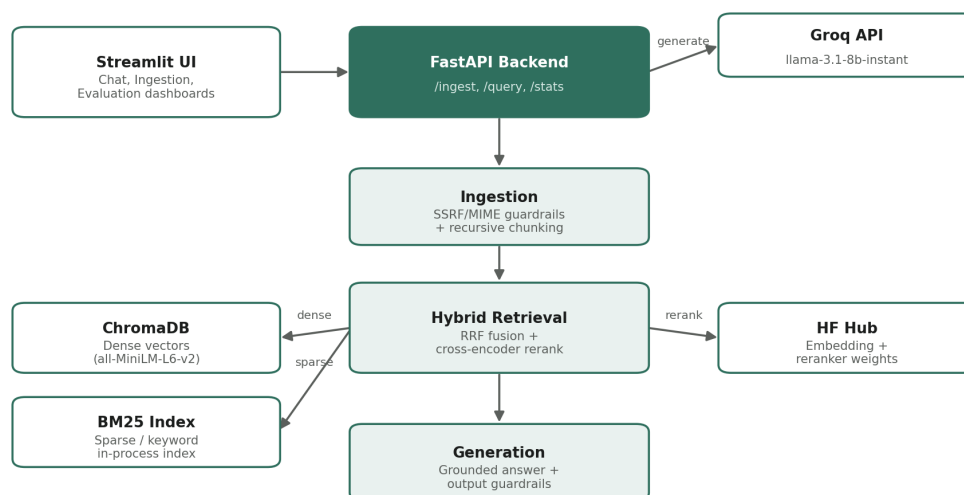


Figure 1. System component diagram: the Streamlit UI talks only to the FastAPI backend, which owns ingestion, hybrid retrieval, and generation.

2.1 Ingestion and Guardrails

Documents are ingested from either a file upload (PDF, DOCX, TXT, CSV) or a URL. File uploads are validated by size and by inspecting file magic bytes rather than trusting the declared MIME type or extension. URL ingestion resolves the target hostname and rejects loopback, private, and link-local addresses, including an explicit block on the AWS metadata IP (169.254.169.254). Because redirects can be used to bypass an initial URL check, each redirect hop is re-validated before being followed, and web crawling respects *robots.txt*. Ingested text is split with a recursive character splitter (500-character chunks, 50-character overlap), chosen over a naive fixed-width splitter because it preferentially breaks on paragraph and sentence boundaries, reducing fragment and mid-sentence-cut rates.

Ingestion Pipeline

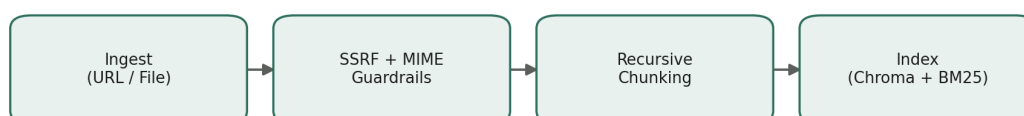


Figure 2. Ingestion pipeline: every document passes through SSRF/MIME guardrails before chunking and indexing.

2.2 Indexing

Each chunk is embedded with a local Sentence-Transformers model (all-MiniLM-L6-v2, Reimers and Gurevych, 2019) and stored in a persistent ChromaDB collection. A parallel BM25 index (Robertson and Zaragoza, 2009) is built over the same chunk set. An observer callback on the embedding store triggers a BM25 rebuild whenever new documents are added, preventing the two indexes from silently drifting out of sync - a failure mode we found undocumented in most hybrid-search tutorials.

2.3 Retrieval Pipeline

At query time, an LLM-based query rewriter first expands the user's natural-language question into a keyword-enriched search query, resolving vague references and surfacing likely synonyms or technical variants (e.g., identifiers, version numbers). Retrieval then proceeds against the rewritten query only; the original question is

preserved separately for the generation stage. The hybrid retriever pulls the top candidates from both the dense (Chroma) and sparse (BM25) indexes and fuses them with Reciprocal Rank Fusion:

$$RRF(d) = \sum_{r \in R} 1 / (k + \text{rank}_r(d))$$

where R is the set of ranked lists (dense, sparse), $\text{rank}_r(d)$ is document d 's rank in list r , and $k = 60$ is a smoothing constant following the original RRF formulation. When a reranker is attached, the retriever widens its pre-fusion candidate pool to $3\times$ the final top- N before fusing, so the cross-encoder has a meaningfully larger set to re-score; without a reranker this reduces to a direct top- N slice, preserving prior behavior. The cross-encoder (cross-encoder/ms-marco-MiniLM-L-6-v2) then scores each (query, document) pair jointly - a strictly more expressive scoring function than the independent dense/sparse scores used for fusion - and the top- N by cross-encoder score are passed to generation. The reranker model is loaded lazily on first use so it never adds latency to application startup.

Query Pipeline



Figure 3. Query pipeline: retrieval runs against the rewritten query; generation and guardrails run against the user's original question.

2.4 Generation and Output Guardrails

The retrieved chunks are formatted into a numbered context block and passed to an LLM (llama-3.1-8b-instant via the Groq API) with a strict grounding prompt: the model is instructed to answer only from the provided context, to cite sources by bracketed index, and to return a fixed refusal string when the context is insufficient. Before generation, the input query is screened for prompt-injection patterns and profanity; after generation, the output is screened for profanity and for hallucinated citations (bracketed indices that do not correspond to any retrieved document). Any guardrail trigger returns a distinct refusal message rather than the model's raw output.

3. Evaluation

3.1 Methodology

We constructed a 105-document synthetic corpus designed to stress-test exact-match recall under semantic distraction. The corpus contains eight target facts (e.g., an order ID, a support ticket number, a config value) alongside 90 adversarial distractor documents that are topically and lexically similar to the targets but do not contain the answer. For each of eight held-out queries, we measure whether the target fact appears in the top results under (a) dense-only retrieval (top-2 of 5) and (b) hybrid retrieval (top-2, RRF-fused). The corpus is built fresh in an isolated temporary database for each evaluation run and does not touch the live application index.

3.2 Results

Retrieval Mode	Hit Rate	Notes
Dense-only (vector search)	8 / 8 (100%)	Target ranked in top 2 on every query
Hybrid (Dense + BM25, RRF-fused)	8 / 8 (100%)	No regression vs. dense-only

Table 1. Dense-only vs. hybrid retrieval hit rate on the 105-document adversarial benchmark (n=8 queries).

3.3 Discussion

At this corpus size, the dense embedding model (all-MiniLM-L6-v2) was already sufficient to rank every target fact in the top two results, so hybrid retrieval shows no measurable lift over dense-only on this benchmark - both saturate at 100%. This is itself informative: it confirms the hybrid architecture is correctly wired and introduces zero regression relative to a simpler dense-only baseline, while remaining positioned to pay off as the corpus scales. The literature on hybrid search (e.g., Cormack et al., 2009) and production RAG deployments generally report hybrid retrieval's advantage growing with corpus size and with query specificity, as dense retrieval is increasingly diluted by semantically similar but non-matching neighbors and as exact-match tokens (IDs, codes) become proportionally rarer in the dense embedding space. Evaluating on a corpus one to two orders of magnitude larger is the most direct way to surface a measurable dense/hybrid gap, and is the first extension we would prioritize.

4. Related Work

AnyDoc builds directly on retrieval-augmented generation as introduced by Lewis et al. (2020), who combine a parametric language model with a non-parametric dense retriever over Wikipedia. The sparse side of our hybrid retriever follows the BM25 probabilistic relevance framework (Robertson and Zaragoza, 2009), and the fusion method follows Reciprocal Rank Fusion (Cormack et al., 2009), which was shown to outperform individual rank-learning methods and more complex fusion schemes on TREC benchmarks without requiring score calibration between retrievers - a practical advantage when combining a cosine-similarity dense retriever with a BM25 sparse retriever whose scores are not directly comparable. Our reranking stage follows Nogueira and Cho (2019), who demonstrated that cross-encoder passage reranking over an initial candidate set substantially improves passage ranking quality relative to representation-based (bi-encoder) retrieval alone. The dense embedding model is a Sentence-Transformers model (Reimers and Gurevych, 2019), chosen for its strong quality-to-latency trade-off in a local, CPU-friendly deployment. We are not aware of prior RAG system write-ups that explicitly discuss SSRF as an ingestion-time attack surface; most security-focused RAG literature instead concentrates on prompt injection and data poisoning, which AnyDoc also mitigates at both the input and output stage.

5. Limitations and Future Work

- The evaluation corpus (105 documents, 8 queries) is small; dense and hybrid retrieval both saturate at 100%, so the benchmark does not yet demonstrate a measurable hybrid advantage. A larger, standard benchmark (e.g., a subset of Natural Questions or HotpotQA) would better separate the two approaches.
- Retrieved chunk metadata (document type, extracted topics) is captured at ingestion time but not yet used to filter or boost retrieval at query time.
- The vector store (ChromaDB, local persistence) and BM25 index are single-node and held in-process; horizontal scaling was out of scope for this iteration.
- Generation is synchronous and non-streamed; streaming would improve perceived latency for longer answers.
- Guardrails use pattern-based prompt-injection detection and a maintained profanity wordlist rather than a dedicated moderation model; a production deployment would likely integrate a hosted moderation API.

6. Conclusion

AnyDoc demonstrates that a hybrid dense/sparse retriever with RRF fusion, cross-encoder reranking, and LLM-based query rewriting can be assembled from open components (ChromaDB, BM25, Sentence-Transformers, a hosted LLM API) into a coherent, testable pipeline, with ingestion-time SSRF hardening addressing an attack

surface that is often overlooked in RAG system design. Evaluation on an adversarial synthetic corpus confirms the pipeline is correctly wired end-to-end; the natural next step is evaluating at a scale where dense-only retrieval begins to degrade, to directly quantify hybrid retrieval's contribution.

References

- Cormack, G. V., Clarke, C. L. A., and Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pp. 758-759.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)* 33.
- Nogueira, R. and Cho, K. (2019). Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.
- Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982-3992.
- Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), pp. 333-389.